1️⃣ First decide the modules

Think **by service**.

For this task:

```
modules
 ├─ vpc
 ├─ ec2
 └─ s3
```

Simple rule:

Module	Contains
VPC	VPC, subnet, IGW, route table
EC2	SG, network interface, EIP, EC2
S3	bucket

🧠 **Memory trick**

```
Network → VPC module
Compute → EC2 module
Storage → S3 module
```

2️⃣ VPC module pattern

Networking always follows this **same order**:

```
VPC
↓
Subnet
↓
Internet Gateway
↓
Route Table
↓
Route Table Association
```

So remember:

```
VPC → Subnet → IGW → Route → Association
```

Then expose values:

```
output "vpc_id"
output "subnet_id"
```

3 EC2 module pattern

Compute setup always follows this order:

```
Security Group
↓
Network Interface
↓
Elastic IP
↓
EC2 Instance
```

So remember:

```
SG → NIC → EIP → EC2
```

4 Module connection rule (VERY IMPORTANT)

Modules **never talk directly**.

They use:

```
Output → Variable
```

Flow:

```
VPC module
   ↓ output
main.tf
   ↓ pass variable
EC2 module
```

Example:

```
module.vpc.subnet_id → var.subnet_id
```

5 Root main.tf pattern

Root file **only connects modules**.

```
provider

module "vpc"

module "s3"

module "ec2" {
  vpc_id
  subnet_id
}
```

Think:

```
main.tf = wiring
modules = actual resources
```

🧠 Ultimate Memory Pattern

When asked this question in exam/interview:

1️⃣ Create modules (vpc / ec2 / s3)

2️⃣ VPC module
VPC → Subnet → IGW → Route → Association

3️⃣ EC2 module
SG → NIC → EIP → EC2

4️⃣ Outputs from VPC
vpc_id
subnet_id

5️⃣ Variables in EC2
vpc_id
subnet_id

6️⃣ main.tf connects modules

🎤 Interview One-Liner

"A Terraform project is organized using modules where networking, compute, and storage resources are separated and connected using module outputs and variables." 🚀